PROYECTO
PROGRAMACIÓN I

1. Descripción del problema

Una ciudad se representa mediante edificios rectangulares que descansan sobre el eje X positivo. Cada edificio se describe con una terna (x_i, x_f, h), donde xi es la abscisa inicial, x_f es la abscisa final y h es la altura. Los edificios pueden estar juntos, separados o superpuestos horizontalmente.

Se requiere desarrollar un programa que permita administrar los edificios registrados, mostrar su representación gráfica y textual, calcular la línea de horizonte visible y determinar el área y el perímetro de la unión de los rectángulos.

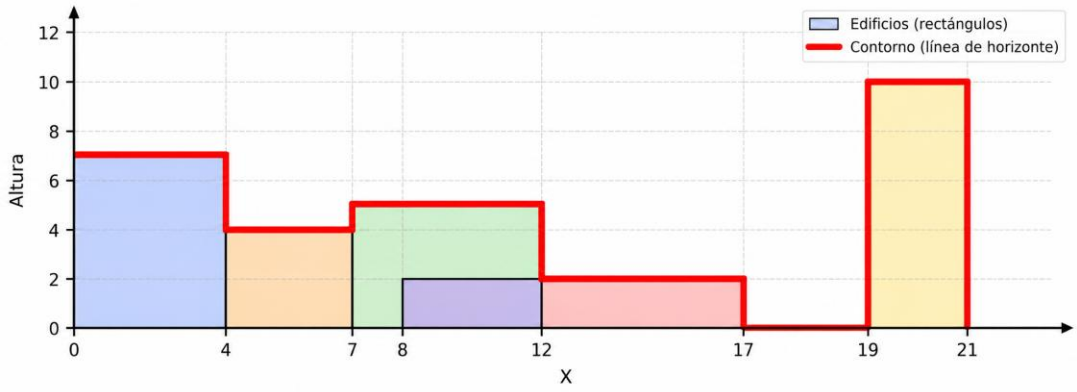
2. Ejemplo de entrada y resultado esperado

Conjunto de edificios:

C = {(0, 4, 7), (4, 7, 4), (7, 12, 5), (8, 17, 2), (19, 21, 10)}

La línea de horizonte se expresa por tramos (inicio, fin, altura). En cada intervalo se toma la altura máxima de los edificios que cubren ese intervalo:

H = {(0, 4, 7), (4, 7, 4), (7, 12, 5), (12, 17, 2), (17, 19, 0), (19, 21, 10)}



Nota: el tramo (17, 19, 0) representa el espacio vacío entre los dos grupos de edificios.

3. Definiciones y reglas de cálculo

3.1 Línea de horizonte

Es el perfil superior visible de la unión de los edificios. Para cada intervalo entre abscisas consecutivas relevantes, la altura del horizonte es el máximo de las alturas de los edificios que cubren dicho intervalo. Los tramos contiguos con la misma altura deben combinarse en un único segmento.

3.2 Área

Se calculará el área de la unión de los rectángulos, sin contar dos veces las zonas superpuestas. Puede obtenerse sumando, para cada intervalo del eje X, su ancho multiplicado por la altura máxima visible en ese intervalo. En el ejemplo, el área es 95 unidades cuadradas.

3.3 Perímetro

Se calculará el perímetro del contorno exterior de la unión de los rectángulos. No se deben contar los bordes internos ocultos por superposición. Si existen componentes separados, se suman los perímetros de cada componente. En el ejemplo, el perímetro total es 74 unidades.

4. Funcionalidades requeridas

- Ingresar un edificio y validar sus datos.
- Modificar los datos de un edificio seleccionado.
- Eliminar un edificio seleccionado.
- Mostrar todos los edificios en forma textual, indicando número, abscisa inicial, abscisa final y altura.
- Mostrar gráficamente los edificios ingresados sobre los ejes X e Y, con escala y etiquetas legibles. El gráfico debe representar cada edificio como un rectángulo, indicando claramente su posición y altura. Debe poder visualizarse la ciudad actual después de registrar los edificios y también después de cargarla desde un archivo.
- Calcular y mostrar la línea de horizonte en forma textual y gráfica. El gráfico del contorno debe distinguirse claramente del gráfico de los edificios y mostrar el perfil superior visible de la ciudad, incluidos los cambios de altura y los tramos vacíos.

La visualización gráfica es obligatoria tanto para la ciudad ingresada (todos los rectángulos de los edificios) como para el resultado del contorno. No basta con imprimir las ternas ni con mostrar únicamente la línea de horizonte. Ambos gráficos deben tener ejes identificados, escala proporcional y valores legibles.

  - Mostrar el área y el perímetro de la unión de los edificios.
  - Actualizar los resultados después de cada alta, modificación o eliminación.
  - Mostrar mensajes claros cuando no existan edificios o se ingresen datos inválidos.

4.1 Guardado y recuperación de ciudades

El programa deberá permitir guardar en un archivo los edificios que conforman la ciudad actual y recuperar esa información en una ejecución posterior. No será suficiente mantener los datos únicamente en memoria durante la ejecución.

- **Guardar ciudad:** solicitar un nombre de archivo o utilizar un nombre predeterminado (por ejemplo, ciudad.txt) y escribir todos los edificios registrados.
- **Cargar ciudad:** leer un archivo previamente guardado, validar su contenido y reconstruir la lista de edificios en memoria. Si el archivo no existe, está vacío o contiene datos inválidos, mostrar un mensaje claro y evitar que el programa falle.
- **Confirmar operaciones:** informar si el guardado o la carga se realizaron correctamente. Si la carga falla, no reemplazar silenciosamente los datos actuales por una lista incompleta.
- Después de cargar una ciudad, permitir mostrar los edificios, modificar o eliminar registros, y volver a calcular el horizonte, el área y el perímetro con los datos recuperados.
- **Formato de archivo propuesto:** un edificio por línea, con tres números separados por espacios: xi xf h. Ejemplo: 0 4 7. El archivo debe contener únicamente los datos necesarios para reconstruir la ciudad; el horizonte, el área y el perímetro se calculan nuevamente al cargar.

- **Manejo de archivos en C**: utilizar las funciones de apertura, lectura/escritura y cierre de archivos vistas en el curso (por ejemplo, fopen, fscanf o fgets, fprintf y fclose). Comprobar siempre que la apertura y las operaciones de lectura/escritura sean correctas.
- Agregar al menú principal, como mínimo, las opciones “Guardar ciudad en archivo” y “Cargar ciudad desde archivo”.

4.2 Persistencia y consistencia de los datos

Las altas, modificaciones y eliminaciones cambian la ciudad en memoria. El estudiante debe indicar al usuario que guarde los cambios para conservarlos después de cerrar el programa. No se exige guardado automático, salvo que se implemente como mejora.

5. Validaciones de entrada

- Las abscisas y la altura deben ser valores numéricos válidos.
- La abscisa inicial debe ser menor que la abscisa final (xi < xf).
- La altura debe ser mayor que cero (h > 0).
- No se aceptarán registros incompletos ni caracteres no numéricos donde se esperen números.
- Los edificios pueden compartir extremos o superponerse; esto no debe considerarse un error.
- Al modificar un edificio, deben validarse nuevamente todos sus datos antes de guardar los cambios.

6. Requisitos de programación

El programa se desarrollará en C, aplicando los contenidos del curso de Programación I. Se deberá:

- Utilizar funciones para dividir el problema en tareas independientes.
- Usar arreglos para almacenar los datos de los edificios, con un límite de capacidad definido y controlado.
- Pasar parámetros a las funciones y utilizar valores de retorno de manera apropiada.
- Separar el registro y la validación de datos, los cálculos y la presentación de resultados.
- Evitar duplicar bloques de código cuando una función pueda reutilizarse.
- Comentar las partes del algoritmo que no sean evidentes y emplear nombres claros para variables y funciones.

7. Pruebas mínimas

Caso	Datos o situación	Resultado que se debe verificar
1	Un solo edificio	Horizonte igual al edificio; área y perímetro correctos.
2	Edificios consecutivos	No debe aparecer un espacio vacío entre ellos.
3	Edificios superpuestos de distintas alturas	Debe mostrarse la altura máxima en cada intervalo.
4	Edificios superpuestos de igual altura	El horizonte debe conservar esa altura sin duplicar área.
5	Edificios separados	El horizonte debe incluir el tramo de altura cero entre componentes.
6	Abscisa inicial igual o mayor que la final	El registro debe rechazarse.
7	Altura cero o negativa	El registro debe rechazarse.
8	Modificar o eliminar un edificio	Horizonte, área y perímetro deben recalcularse.
9	Lista vacía	El programa debe informar que no hay edificios y evitar cálculos inválidos.
10	Guardar la ciudad, cerrar el programa y volver a ejecutarlo	El archivo debe conservar todos los edificios y sus datos.
11	Cargar una ciudad guardada previamente	Se deben recuperar los edificios y recalcular horizonte, área y perímetro.
12	Intentar cargar un archivo inexistente o con datos inválidos	Mostrar un mensaje de error sin provocar un fallo del programa ni cargar datos incompletos.
13	Modificar o eliminar edificios después de cargar un archivo	Los resultados deben actualizarse; al guardar nuevamente, el archivo debe reflejar los cambios.

8. Entregables

- Código fuente completo (.c) y los archivos adicionales necesarios para compilarlo. Incluya el manejo de archivos para guardar y cargar ciudades.
- Programa ejecutable o instrucciones claras para compilar y ejecutar.
- Informe que describa la solución, las funciones implementadas y la estrategia de cálculo.
- Evidencias de ejecución que muestren registro, modificación, eliminación, validaciones y ambas representaciones gráficas:
    1. los edificios ingresados como rectángulos sobre los ejes X e Y y
    2. el contorno o línea de horizonte resultante. Incluya también una prueba de guardado, cierre y recuperación de una ciudad, mostrando los gráficos y los cálculos después de cargarla.

- Resultados de las pruebas mínimas, incluyendo los datos utilizados y los resultados obtenidos.

9. Criterios de evaluación (20 puntos)

Criterio	Puntaje
Gestión de edificios: registrar, modificar y eliminar	3
Validación de entradas y manejo de errores	2
Cálculo correcto de la línea de horizonte, área perímetro	5
Representación gráfica y salida textual	2
Guardado y carga de ciudades mediante archivos	2
Organización del código, funciones y evidencias de pruebas	2
Entregables y sustentación	4

Total: 20 puntos

10. Consideraciones finales

La solución debe funcionar con diferentes conjuntos de edificios, no solamente con los datos del ejemplo. La corrección se evaluará con casos adicionales. Las unidades de área y perímetro dependen de las unidades utilizadas para las coordenadas y alturas.

Ejemplo del contenido de ciudad.txt:

```
0 4 7
4 7 4
7 12 5
8 17 2
19 21 10
```

Cada línea representa un edificio en el orden x_i, x_f y h. Al cargar el archivo, el programa debe validar cada línea y recalcular todos los resultados a partir de los edificios leídos.

11. Análisis obligatorio de los casos geométricos

Antes de implementar el algoritmo, el equipo debe analizar cómo se comporta el contorno ante diferentes configuraciones de edificios. No basta con que el programa funcione con el ejemplo del enunciado: debe resolver los casos siguientes, documentar la estrategia utilizada y demostrarla mediante pruebas. Los casos pueden combinarse en una misma ciudad.

- **Edificio único.** La ciudad contiene un solo rectángulo. El horizonte debe coincidir con su parte superior y sus límites; el área y el perímetro deben ser correctos.
- **Edificios consecutivos que se tocan.** El final de un edificio coincide con el inicio del siguiente. Determinar si cambia la altura y evitar introducir un espacio vacío inexistente.
- **Edificios separados.** Existe una distancia positiva entre dos edificios o grupos. El contorno debe descender hasta altura cero y volver a subir al comenzar el siguiente grupo.
- **Superposición parcial con alturas diferentes.** Dos o más edificios cubren intervalos comunes, pero tienen distintas alturas. En cada intervalo debe prevalecer la altura máxima.
- **Superposición parcial con la misma altura.** Varios edificios se superponen o coinciden en altura. Evitar segmentos duplicados y contabilizar el área de la unión una sola vez.
- **Edificio completamente oculto.** Un edificio queda dentro del intervalo horizontal de otro más alto y no aporta ningún tramo al horizonte. El programa debe conservarlo en el registro, pero no inventar una contribución visible.
- **Edificio parcialmente oculto.** Una parte del edificio queda por debajo de otro edificio más alto y otra parte queda expuesta. El horizonte solo debe incluir la parte que resulte visible.
- **Varios edificios con el mismo inicio.** Dos o más edificios comienzan en la misma abscisa. El algoritmo debe determinar la altura visible correcta desde ese punto.
- **Varios edificios con el mismo final.** Dos o más edificios terminan en la misma abscisa. El algoritmo debe actualizar correctamente la altura visible después de ese límite.
- **Final e inicio compartidos por varios edificios.** En una misma coordenada terminan algunos edificios y comienzan otros. Procesar todos los cambios de esa coordenada de forma coherente, sin crear tramos de ancho cero.
- **Cambios sucesivos de altura.** El horizonte sube y baja varias veces a lo largo de la ciudad. Cada segmento debe abarcar un intervalo de ancho positivo y tener la altura máxima correspondiente.
- **Múltiples grupos separados.** La ciudad contiene tres o más grupos desconectados. Representar cada espacio vacío entre grupos y calcular el contorno de todos los componentes.
- **Edificios anidados.** Un edificio de menor anchura queda completamente dentro del intervalo de uno más ancho. Analizar si su altura modifica o no el horizonte.
- **Alturas crecientes y decrecientes.** La altura de los edificios aumenta y luego disminuye, o viceversa. Verificar cada transición y la eliminación de puntos redundantes.
- **Edificios idénticos o duplicados.** Se registran dos edificios con las mismas coordenadas y altura. La aplicación debe definir y documentar si permite duplicados; en cualquier caso, no debe duplicar el área de la unión.
- **Coordenadas repetidas y valores límite.** Varios edificios comparten coordenadas de inicio o fin. Comprobar límites de capacidad de los arreglos y evitar errores de índice.
- **Conjunto vacío.** No hay edificios registrados o el archivo cargado no contiene edificios. Informar la situación y evitar cálculos o gráficos inválidos.
- **Cambios en la ciudad.** Después de insertar, modificar o eliminar un edificio, recalcular el horizonte, el área y el perímetro. Verificar que no permanezcan resultados antiguos.
- **Guardar y recuperar.** Guardar una ciudad con superposiciones y espacios vacíos, cerrar el programa, cargarla en otra ejecución y comprobar que se reconstruyen los mismos edificios y resultados.

- **Archivo incorrecto o incompleto.** El archivo puede no existir, estar vacío, contener líneas incompletas, valores no numéricos o edificios geométricamente inválidos. Informar el error y evitar aceptar silenciosamente datos parciales.

11.1 Qué debe comprobarse en cada caso

- Conjunto de entrada: listar las ternas (x_i, x_f, h) utilizadas.
- Contorno esperado: indicar los segmentos (inicio, fin, altura), incluidos los intervalos de altura cero cuando existan espacios vacíos.
- Área esperada: calcular la unión de los rectángulos sin contar dos veces las zonas superpuestas.
- Perímetro esperado: calcular el contorno exterior de la unión; no sumar simplemente los perímetros individuales ni confundirlo con la longitud de la línea de horizonte.
- Gráficos: mostrar los edificios como rectángulos y el contorno resultante en una representación claramente identificada.
- Conclusión: explicar por qué el resultado es correcto y qué condición del algoritmo se verifica.

12. Explicación y documentación obligatoria de los algoritmos

El informe debe explicar y documentar los algoritmos que el equipo haya implementado. No es suficiente incluir el código fuente ni describir el programa únicamente mediante capturas de pantalla. Cada algoritmo principal y cada caso especial relevante deben explicarse con palabras propias.

12.1 Ficha de documentación para cada algoritmo

- **Nombre y objetivo:** qué problema resuelve el algoritmo y cuándo se utiliza.
- **Entradas:** datos que recibe, su significado y las restricciones que deben cumplir.
- **Salidas:** resultados que produce y el formato en que se representan.
- Precondiciones y poscondiciones: qué debe cumplirse antes de ejecutarlo y qué garantiza al finalizar.
- **Idea y lógica paso a paso:** explicar cómo se transforma la entrada en el resultado, incluyendo las decisiones y los ciclos utilizados.
- **Pseudocódigo:** representar la lógica antes o junto con el código C.
- **Funciones involucradas:** indicar nombre, responsabilidad, parámetros, valor de retorno y relación con otras funciones.
- **Trazado manual**: desarrollar al menos un ejemplo paso a paso, mostrando cómo cambian las variables importantes y cómo se obtiene cada segmento del contorno.
- **Caso límite:** explicar qué ocurre con superposiciones, alturas iguales, extremos coincidentes, edificios ocultos o espacios vacíos, según corresponda.
- **Validación:** describir cómo se comprobó el resultado con casos de prueba y cuál era el resultado esperado.
- **Complejidad básica:** identificar los ciclos principales y explicar, de forma acorde con el curso, cómo aumenta el trabajo cuando crece el número de edificios.
- **Limitaciones:** señalar las restricciones de capacidad, las decisiones de diseño y los casos no contemplados, si los hubiera.

12.2 Algoritmos que deben documentarse como mínimo

- Registrar y validar un edificio.
- Modificar y eliminar un edificio sin corromper los datos almacenados.

- Calcular la línea de horizonte, explicando cómo se obtiene la altura visible en cada intervalo y cómo se eliminan segmentos redundantes.
- Calcular el área de la unión de edificios sin contar dos veces las superposiciones.
- Calcular el perímetro del contorno exterior de la unión, considerando bordes compartidos, lados ocultos y componentes separados.
- Generar el gráfico de los rectángulos y el gráfico independiente del horizonte.
- Guardar una ciudad y cargarla desde un archivo, incluyendo la validación de errores.
- Actualizar los cálculos después de insertar, modificar, eliminar o cargar edificios.

12.3 Evidencia mínima que debe acompañar al informe

- Tabla de pruebas con identificador, datos de entrada, resultado esperado, resultado obtenido y estado (correcto/incorrecto).
- Al menos un trazado manual completo del algoritmo de horizonte con superposición de edificios.
- Una prueba con edificios separados, una con alturas iguales y una con un edificio total o parcialmente oculto.
- Capturas o evidencias de ambos gráficos para una misma ciudad.
- Evidencia de guardar, cerrar, volver a ejecutar, cargar la ciudad y obtener los mismos edificios y resultados.
- Explicación de cualquier diferencia entre el resultado esperado y el obtenido, junto con la corrección realizada.
- La documentación debe corresponder al código realmente entregado. Si el equipo cambia un algoritmo durante el desarrollo, deberá actualizar el pseudocódigo, la explicación y las pruebas para que los tres coincidan.